

My bot is failing due to it relying on model GPT-4.10-nano, a model notorious for its mathematical shortcomings. As a “nano” model, it is more lightweight and geared towards token prediction. This means that should math problems get more abstract and longer, it will continue to rely on its statistical predictions and guess the most likely next token (digit), culminating in a very wrong answer. This issue is compounded by the bot’s iterative design. Once the bot produces an incorrect final product for its iteration, this answer will still be used as the base in the following iteration. Meaning that from the first error onwards, though the python method is informing us of the correct answer in each iteration, the bot will continue calculations using numbers that the code can deduce are incorrect. As a result, the bot will progressively drift further and further away from a correct final answer. The more iterations that are prompted by the user at run time, the more likely it becomes that the base numbers will get too large for the model to handle, and that each mistake is compounded with the next. All of these factors culminate in a bot that is prone to errors and cannot self-correct.

To make the bot more interesting, I could have it automatically take the python product value as its base for the following iteration. This would solve the problem of the bot’s inability to recover from incorrect calculations and might even result in it succeeding after a previous failure. However, once the bot makes an error, it’s likely that the calculations are already too long for the model to succeed. This, in turn with the entertainment of the bot’s increasingly self-deprecating comments per successive failures, is why I chose not to implement this potential change.